KOO XUAN (A23CS0300)

SECJ2013-01 2425-1 LAB 2

```
#include <iostream>
using namespace std;

void bubbleSort(int data[], int listSize)
{
    // implement the bubble sort algorithm


} // end bubble sort

void iBubbleSort(int data[], int listSize)
{
    // implement the improved bubble sort algorithm


} // end improved bubble sort

void selectionSort(int data[], int n)
{
    // implement the selection sort algorithm


} // end selection sort

void swap(int& x, int& y)
{

} // end swap

void insertionSort(int data[])
{
    // implement the insertion sort algorithm


} // end insertion sort

int main(){
    int dataArrayA[25] = { };
    int dataArrayB[25] = { };

    // Implement your code to call bubble sort, improved bubble sort,
    // selection sort, and insertion sort functions seperately.


}
```

SECJ2013-01 2425-1 LAB 2

Complete program above to sort 25 integers below into ascending order using simple sort algorithms: Bubble Sort, Insertion Sort and Selection Sort. For each sorting technique learned in class, modify the algorithms so that the programs are able to count and print the number of passes, the number of data comparisons and the number of data swapping that take place in the sorting process.

a. Run the programs on the following list of integer values. You can initialize the data in the array declarations.

int dataArrayA[25] = {.....};

int dataArrayB[25] = {..... };

100	50	88	30	60	45	25	12	10	5	98	15	65	55	45	70	20	90	66	22	120	48	35	85	5

dataArrayA

5	8	30	25	35	40	42	50	55	22	24	66	70	75	78	80	85	90	100	118	98	120	122	121	121

dataArrayB

b. dataArrayA is an example of a worst case data — totally unsorted list, while dataArrayB is an example of a best case data — almost sorted list. Analyze the output displayed from each sorting technique to find the total number of passes, number of data comparisons and number of data swapping. Make conclusions on which sorting technique is the best for the worst case data and which technique is the best for best case data. Which technique has performance that does not depend on the initial arrangement of data?

Ans :

For the worst case which is dataArrayA, Bubble Sort requires the maximum number of comparisons and a large number of swaps, making it inefficient for unsorted data. Improved Bubble Sort demonstrating excellent efficiency when data is already sorted after an early pass. Selection Sort making it predictable but not ideal for worst-case scenarios. Insertion Sort is very efficient in terms of comparisons and no swaps in the worst case with approach for only small data changes.

For the best case which is dataArrayB, Bubble Sort lacks optimization for already sorted or nearly sorted data. Improved Bubble Sort stops after the first pass as the array is already sorted is the best performance among all techniques in the best case. Selection Sort minimal swaps make it predictable but inefficient compared to other techniques.

Insertion Sort completes sorting with minimal comparisons and no swaps shows it's highly efficient in the best case.

To conclude, insertion sort is the most suitable technique for worst case data whereas improved bubble sort is the most suitable technique for best case data. Selection sort has performance that does not depend on the initial arrangement of data.

c. Fill in the following table to help you discuss the performance analysis.

Technique	**Case**	**No of Comparisons**	**No of Swaps**	**No of Passes**
Conventional Bubble Sort	Worst Case	300	154	24
	Best Case	300	19	24
Discussion: Discuss the performance analysis for Conventional Bubble Sort				
Improved Bubble Sort	Worst Case	24	0	1
	Best Case	24	0	1
Discussion: Discuss the performance analysis for Improved Bubble Sort				
Selection Sort	Worst Case	300	2	24
	Best Case	300	1	24
Discussion: Discuss the performance analysis for Selection Sort				
Insertion Sort	Worst Case	24	0	24
	Best Case	24	0	24
Discussion: Discuss the performance analysis for Insertion Sort				